

Project Presentation of CO-OP Project at Industry (Module-2) (22CS421)

On

Security Assurance and Compliance Verification in AI-Generated Software Systems

Abhayjeet Singh, Simran Jaswal,
Abhishek Chaudhary, Chirag Mehta
2210990031, 2210992383,
2210991161, 2210991463

Supervised By

Dr. Preeti Saini

Department of Computer Science and Engineering,
Chitkara University, Punjab

Problem Statement



AI-generated code contains hidden vulnerabilities undetected by traditional software testing methods.

Functional Correctness

Generated code performs intended tasks without errors

Hidden Vulnerabilities

Security flaws remain undetected during normal software testing

AI-Assisted Code Security

Automated tools identify weaknesses in generated application code

Secure Code Generation

AI models generate safer and reliable software systems



Research Objectives

Transform AI-generated code analysis from functional validation to proactive security assessment.



Vulnerability Detection

Detect hidden vulnerabilities in generated code



OWASP Classification

Categorize vulnerabilities using OWASP security standards



Secure Code Analysis

Analyze AI-generated code for security weaknesses



Comparative Evaluation

Compare security across multiple AI coding tools

Proposed Solution



Intelligent Security Evaluation Framework

Our approach combines AI-generated code analysis with automated security assessment to identify vulnerabilities and improve secure software development practices.

- Automated vulnerability detection and classification
- OWASP Top 10 security analysis
- AI-generated code risk assessment
- Multi-layer security review framework



System Architecture

End-to-end pipeline from AI code generation to security analysis and vulnerability visualization.



Prompt Dataset

Natural language development task prompts



AI Code Generation

LLM-generated Python application code



Static Analysis

Automated vulnerability detection using SAST tools



Security Review

OWASP-based expert vulnerability assessment



Result Dashboard

Real-time vulnerability analysis and visualization

01

Prompt Creation

Design realistic software development task prompts

02

AI Code Generation

Generate code using multiple LLM coding assistants

03

Code Collection

Store generated Python artifacts for analysis

04

Vulnerability Detection

Identify security flaws using automated SAST tools

05

Result Analysis

Perform OWASP-based manual vulnerability assessment

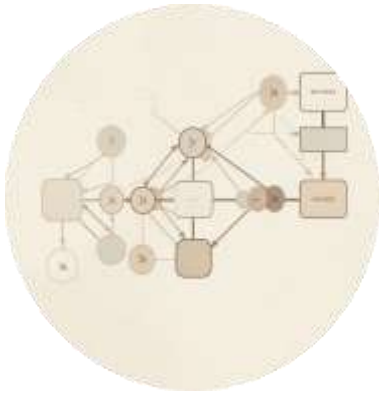
06

Result Analysis

Analyze vulnerability patterns and severity levels

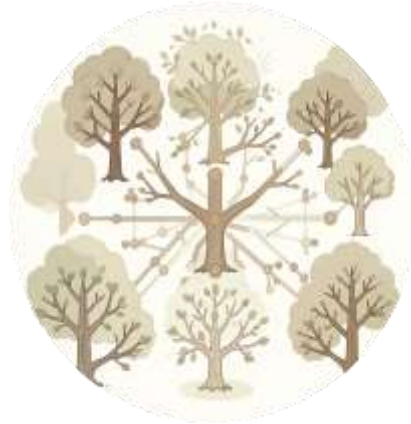
Vulnerability Detection Results

143 unique vulnerabilities identified across 90 AI-generated Python artifacts using a 3-layer detection pipeline.



A03 Injection

32.2% of all findings —
most prevalent category



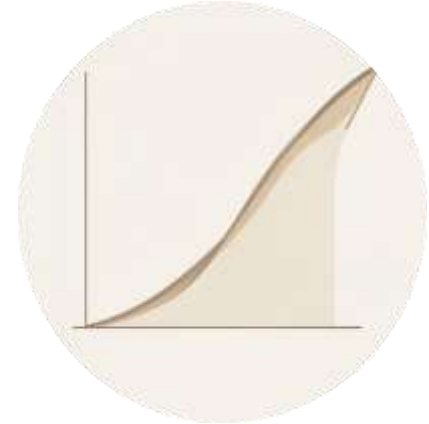
A02 Cryptographic Failures

20.3% — weak hashing
in auth artifacts



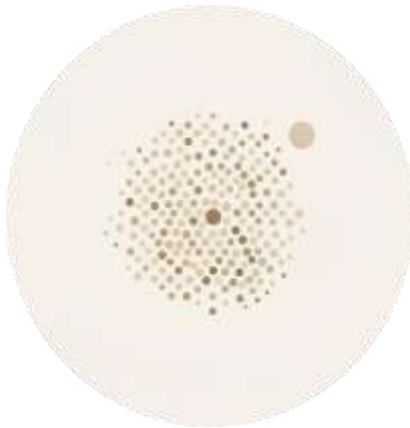
A07 Auth. Failures

18.9% — session & access
control issues



A01 Broken Access Control

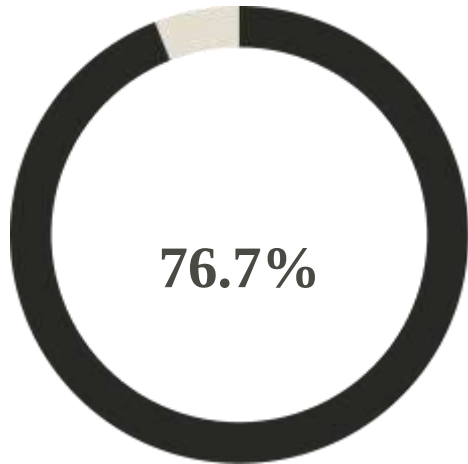
13.3% — enforcement gaps in generated



A05 Misconfiguration

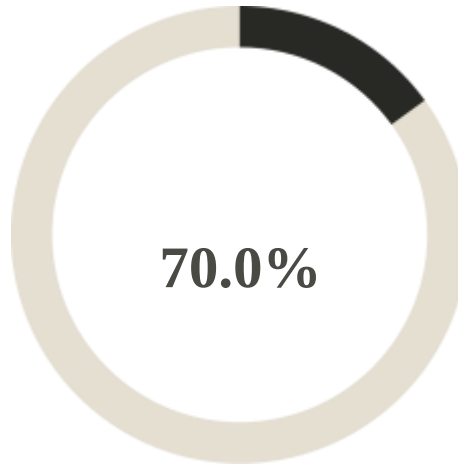
8.4% — debug mode, permissive CORS, disabled SSL

Tool Comparison Results



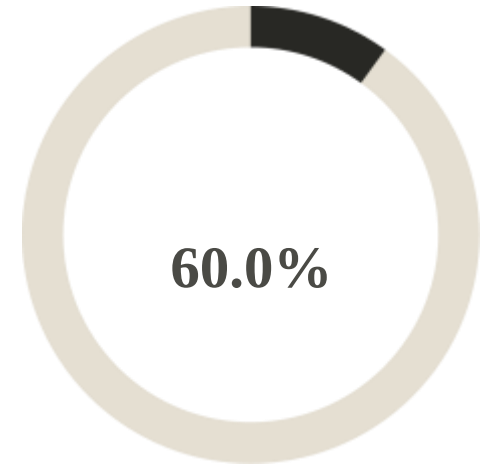
GPT-4o CVR

Highest critical vulnerability rate across all
three tools



GitHub Copilot CVR

SQL injection was the most prevalent
subtype across all tools



Claude 3.5 Sonnet CVR

Lowest rate — best relative security
posture among tested tools



Practices for Secure AI-Augmented Development

AI-generated code cannot be safely deployed without structured security validation at multiple stages of the SDLC.

- Mandate SAST gates in CI/CD to block HIGH/CRITICAL findings
- Use security-aware prompt engineering (parameterised queries, approved crypto libs)
- Apply full manual review to auth, access control, and crypto components
- Provide AI tools with RAG access to vetted secure code templates

Key Achievements

- Improved Security Detection
- Enhanced Vulnerability Analysis
- Reliable Risk Assessment

Future Enhancements

- Real-Time Monitoring
- Advanced AI Detection
- Multi-Language Support

Thank You